

SHETH LUJ & MV COLLEGE

PRACTICAL NO 5

AIM : Key Exchange using Diffie-Hellman

Implement the Diffie-Hellman key exchange algorithm to securely exchange keys between two entities over an insecure network.

NORMAL CODE :

```
# Diffie-Hellman Key Exchange Algorithm

# Public parameters
prime_number = 23
generator = 5

# Private keys selected by Ram and Sham
ram_private = 6
sham_private = 15

# Generate public keys
ram_public = (generator ** ram_private) % prime_number
sham_public = (generator ** sham_private) % prime_number

print("Diffie-Hellman Key Exchange")
print("=====")
print("Prime Number:", prime_number)
print("Generator:", generator)

print("\nRam's Private Key:", ram_private)
print("Ram's Public Key:", ram_public)

print("\nSham's Private Key:", sham_private)
print("Sham's Public Key:", sham_public)

# Calculate the shared secret key
ram_shared_key = (sham_public ** ram_private) % prime_number
sham_shared_key = (ram_public ** sham_private) % prime_number

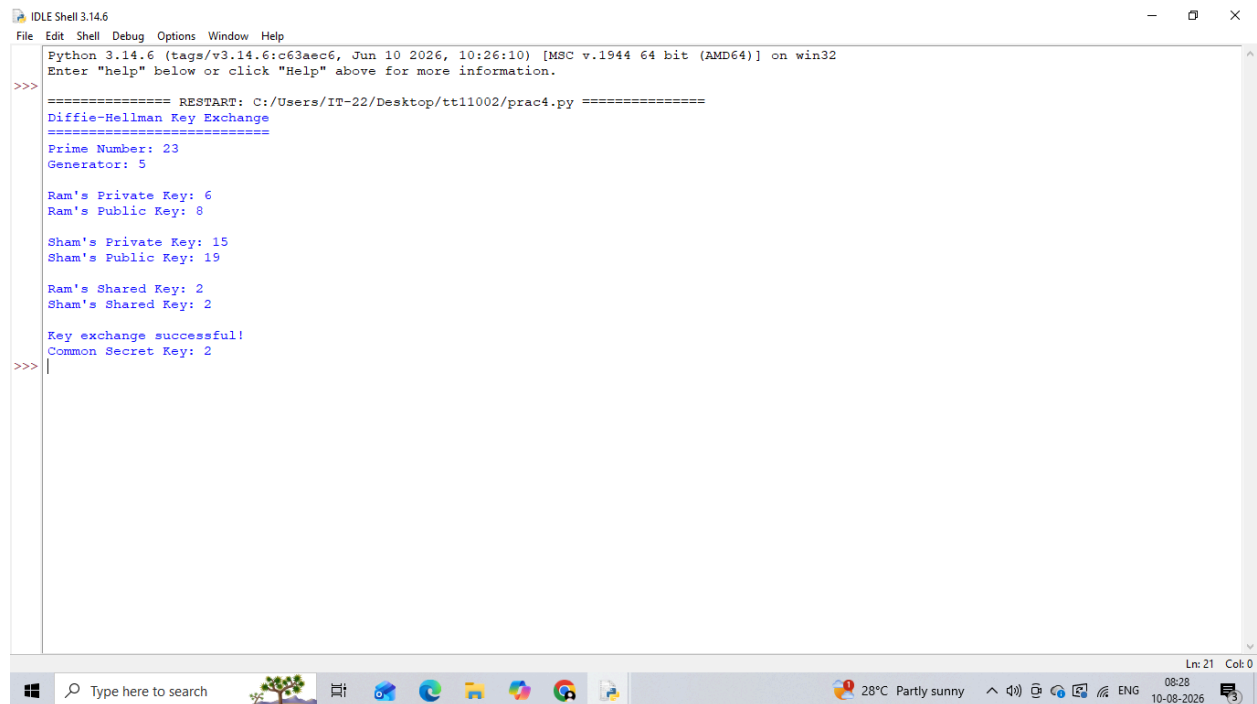
print("\nRam's Shared Key:", ram_shared_key)
print("Sham's Shared Key:", sham_shared_key)

# Check whether both keys are identical
```

SHETH LUJ & MV COLLEGE

```
if ram_shared_key == sham_shared_key:
    print("\nKey exchange successful!")
    print("Common Secret Key:", ram_shared_key)
else:
    print("\nKey exchange failed!")
```

OUTPUT :



```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/IT-22/Desktop/tt11002/prac4.py =====
Diffie-Hellman Key Exchange
=====
Prime Number: 23
Generator: 5

Ram's Private Key: 6
Ram's Public Key: 8

Sham's Private Key: 15
Sham's Public Key: 19

Ram's Shared Key: 2
Sham's Shared Key: 2

Key exchange successful!
Common Secret Key: 2
>>>
```

SHETH LUJ & MV COLLEGE

GUI CODE :

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
def generate_key():
```

```
    try:
```

```
        # Get input values
```

```
        p = int(prime_entry.get())
```

```
        g = int(generator_entry.get())
```

```
        ram_private = int(ram_private_entry.get())
```

```
        sham_private = int(sham_private_entry.get())
```

```
        # Calculate public keys
```

```
        ram_public = pow(g, ram_private, p)
```

```
        sham_public = pow(g, sham_private, p)
```

```
        # Calculate shared keys
```

```
        ram_shared = pow(sham_public, ram_private, p)
```

```
        sham_shared = pow(ram_public, sham_private, p)
```

```
        # Display public keys
```

```
        ram_public_entry.config(state="normal")
```

```
        ram_public_entry.delete(0, tk.END)
```

```
        ram_public_entry.insert(0, ram_public)
```

```
        ram_public_entry.config(state="readonly")
```

```
        sham_public_entry.config(state="normal")
```

```
        sham_public_entry.delete(0, tk.END)
```

```
        sham_public_entry.insert(0, sham_public)
```

```
        sham_public_entry.config(state="readonly")
```

```
        # Display shared keys
```

```
        ram_shared_entry.config(state="normal")
```

```
        ram_shared_entry.delete(0, tk.END)
```

```
        ram_shared_entry.insert(0, ram_shared)
```

```
        ram_shared_entry.config(state="readonly")
```

```
        sham_shared_entry.config(state="normal")
```

```
        sham_shared_entry.delete(0, tk.END)
```

SHETH LUJ & MV COLLEGE

```
sham_shared_entry.insert(0, sham_shared)
sham_shared_entry.config(state="readonly")

# Check result
if ram_shared == sham_shared:
    result_label.config(
        text=f"Key Exchange Successful\nShared Key: {ram_shared}",
        fg="#16803A"
    )
else:
    result_label.config(
        text="Key Exchange Failed",
        fg="#C62828"
    )

except ValueError:
    messagebox.showerror(
        "Error",
        "Please enter valid numbers."
    )

# Main Window
window = tk.Tk()
window.title("Diffie-Hellman Key Exchange")
window.geometry("600x560")
window.configure(bg="#F4F6F8")
window.resizable(False, False)

# Title
title = tk.Label(
    window,
    text="Diffie-Hellman Key Exchange",
    font=("Arial", 20, "bold"),
    bg="#F4F6F8",
    fg="#263238"
)
title.pack(pady=20)
```

SHETH LUJ & MV COLLEGE

```
# Public Parameters
parameter_frame = tk.LabelFrame(
    window,
    text="Public Parameters",
    font=("Arial", 11, "bold"),
    bg="#F4F6F8",
    fg="#37474F",
    padx=20,
    pady=10
)
parameter_frame.pack(padx=30, fill="x")

tk.Label(
    parameter_frame,
    text="Prime Number (p):",
    bg="#F4F6F8"
).grid(row=0, column=0, padx=10, pady=7)

prime_entry = tk.Entry(
    parameter_frame,
    width=15,
    justify="center"
)
prime_entry.grid(row=0, column=1)
prime_entry.insert(0, "23")

tk.Label(
    parameter_frame,
    text="Generator (g):",
    bg="#F4F6F8"
).grid(row=0, column=2, padx=10)

generator_entry = tk.Entry(
    parameter_frame,
    width=15,
    justify="center"
)
generator_entry.grid(row=0, column=3)
```

SHETH LUJ & MV COLLEGE

```
generator_entry.insert(0, "5")
```

```
# Users Frame
```

```
users_frame = tk.Frame(  
    window,  
    bg="#F4F6F8"  
)  
users_frame.pack(pady=20)
```

```
# Ram Frame
```

```
ram_frame = tk.LabelFrame(  
    users_frame,  
    text="Ram",  
    font=("Arial", 11, "bold"),  
    bg="#E3F2FD",  
    fg="#1565C0",  
    padx=15,  
    pady=10  
)  
ram_frame.grid(row=0, column=0, padx=10)
```

```
tk.Label(  
    ram_frame,  
    text="Private Key:",  
    bg="#E3F2FD"  
)grid(row=0, column=0, pady=6)
```

```
ram_private_entry = tk.Entry(  
    ram_frame,  
    width=15,  
    justify="center"  
)  
ram_private_entry.grid(row=0, column=1)  
ram_private_entry.insert(0, "6")
```

```
tk.Label(  

```

SHETH LUJ & MV COLLEGE

```
    ram_frame,  
    text="Public Key:",  
    bg="#E3F2FD"  
)grid(row=1, column=0, pady=6)
```

```
ram_public_entry = tk.Entry(  
    ram_frame,  
    width=15,  
    justify="center"  
)  
ram_public_entry.grid(row=1, column=1)  
ram_public_entry.config(state="readonly")
```

```
tk.Label(  
    ram_frame,  
    text="Shared Key:",  
    bg="#E3F2FD"  
)grid(row=2, column=0, pady=6)
```

```
ram_shared_entry = tk.Entry(  
    ram_frame,  
    width=15,  
    justify="center"  
)  
ram_shared_entry.grid(row=2, column=1)  
ram_shared_entry.config(state="readonly")
```

```
# Sham Frame  
sham_frame = tk.LabelFrame(  
    users_frame,  
    text="Sham",  
    font=("Arial", 11, "bold"),  
    bg="#FCE4EC",  
    fg="#C2185B",  
    padx=15,  
    pady=10  
)  
sham_frame.grid(row=0, column=1, padx=10)
```

SHETH LUJ & MV COLLEGE

```
tk.Label(  
    sham_frame,  
    text="Private Key:",  
    bg="#FCE4EC"  
).grid(row=0, column=0, pady=6)
```

```
sham_private_entry = tk.Entry(  
    sham_frame,  
    width=15,  
    justify="center"  
)  
sham_private_entry.grid(row=0, column=1)  
sham_private_entry.insert(0, "15")
```

```
tk.Label(  
    sham_frame,  
    text="Public Key:",  
    bg="#FCE4EC"  
).grid(row=1, column=0, pady=6)
```

```
sham_public_entry = tk.Entry(  
    sham_frame,  
    width=15,  
    justify="center"  
)  
sham_public_entry.grid(row=1, column=1)  
sham_public_entry.config(state="readonly")
```

```
tk.Label(  
    sham_frame,  
    text="Shared Key:",  
    bg="#FCE4EC"  
).grid(row=2, column=0, pady=6)
```

```
sham_shared_entry = tk.Entry(  
    sham_frame,
```


SHETH LUJ & MV COLLEGE

```
        width=15,  
        justify="center"  
    )  
    sham_shared_entry.grid(row=2, column=1)  
    sham_shared_entry.config(state="readonly")
```

```
# Generate Button  
generate_button = tk.Button(  
    window,  
    text="Generate Shared Key",  
    command=generate_key,  
    font=("Arial", 11, "bold"),  
    bg="#1976D2",  
    fg="white",  
    activebackground="#1565C0",  
    activeforeground="white",  
    padx=20,  
    pady=8,  
    relief="flat"  
)  
generate_button.pack(pady=5)
```

```
# Result  
result_label = tk.Label(  
    window,  
    text="Enter values and click Generate Shared Key",  
    font=("Arial", 11, "bold"),  
    bg="#F4F6F8",  
    fg="#546E7A"  
)  
result_label.pack(pady=15)
```

```
window.mainloop()
```

SHETH LUJ & MV COLLEGE

OUTPUT :

